

INTERVIEW PREPARATION PACKAGE

Staff / Senior Machine Learning Engineer

System Design Round · ML Systems + AI Agents (LLM / RAG)

Target Company: Xero

10 ML & Agent design questions

Whiteboard-ready architectures

Xbot RAG deep-dive

Prepared by your interview coach — 10 yrs hiring Staff/Senior ML at MANGOS, mid-size tech & startups
Focus: solution architecture · production readiness · research↔production boundary · technical leadership

0 · How This Round Is Scored (Read First)

This round is **not** about model internals, loss functions, or the latest paper. The brief is explicit: you will be given a problem and asked to design a **production ML / AI system end-to-end**, focusing on components, infrastructure, data, and the interfaces between the **research** and **production** environments. The interviewer will run a **whiteboard session**. Your job is to *drive the board*, not wait to be prompted.

What Xero is really testing at Staff/Senior level

Signal	What "good" looks like
Architecture breadth	You name every layer — ingestion, storage, feature/index, training/indexing, serving, monitoring, feedback — and explain why each exists and how they connect.
Production readiness	Latency/throughput SLOs, scaling, failover, rollout strategy (shadow → canary → full), cost, security/PII, and on-call observability.
Research↔Production boundary	You can articulate the handoff: a model/training contract, a feature/embedding contract, reproducibility, offline vs online parity, and how a researcher safely ships to prod.
Technical leadership	You state assumptions, make explicit trade-offs, defend decisions, simplify the v1, and describe how you'd de-risk and sequence delivery across a team.
Evaluation rigor	You define offline + online metrics, ground-truth strategy, A/B framework, and guardrail metrics — before talking about "the model".

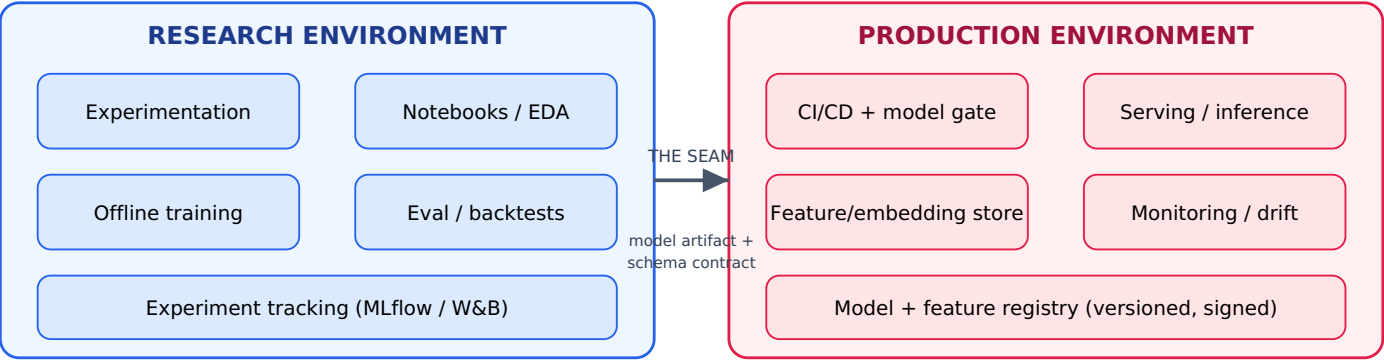
The 6-step framework — use this on every question

- Clarify & scope.** Restate the problem, ask 2-3 sharp questions (scale, latency, batch vs real-time, accuracy bar, who consumes output), then state assumptions out loud.
- Define success.** Business metric → ML metric → guardrail metric → SLOs (p99 latency, QPS, availability, cost/req).
- Data & contracts.** Sources, volume, labels/ground truth, freshness, PII, and the schema contract between research and prod.
- Draw the architecture.** Left-to-right: sources → processing → store → train/index → serve → monitor → feedback. Talk while you draw.
- Production concerns.** Scaling, rollout, fallback, versioning/rollback, retraining triggers, observability, security.
- Trade-offs & v2.** Call out what you deliberately deferred, the biggest risk, and how you'd evolve it.

Coaching tip: Spend the first 3-4 minutes on steps 1-2 *before* drawing. Staff candidates who jump straight to boxes-and-arrows lose points for not framing. Narrate trade-offs continuously — "I'll use a feature store here rather than recomputing because training/serving skew is the #1 silent killer in prod ML."

The research↔production boundary (your differentiator)

Most candidates draw a model in a box. Staff candidates draw the **seam** between two worlds and govern it:



The seam = a versioned model artifact + an explicit feature/embedding schema contract + reproducible training spec. Govern it and you sound like Staff.

1 · Top 5 ML System Design Questions

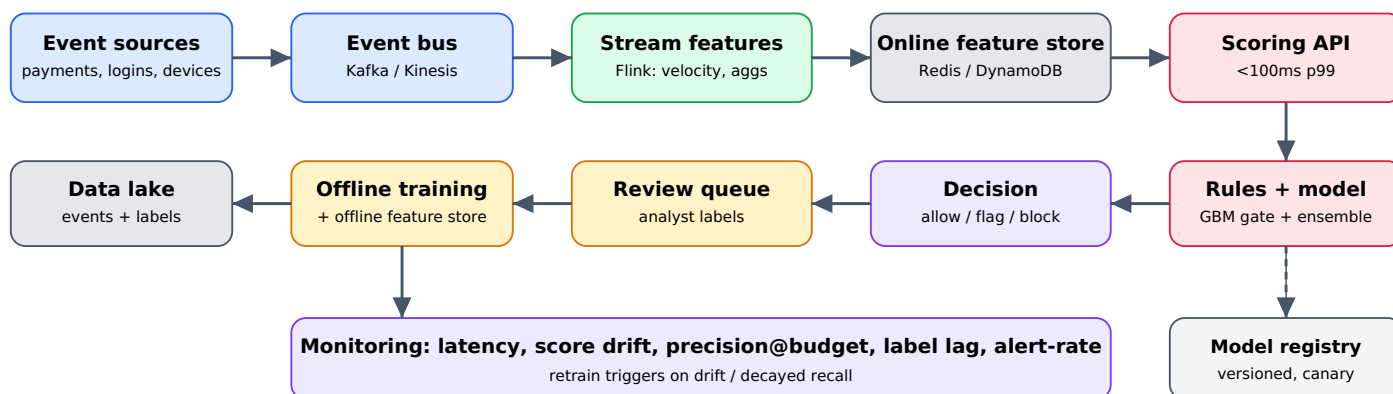
Each answer is framed the way you should answer on the whiteboard: scope → metrics → data/contracts → architecture → production → trade-offs. Diagrams are deliberately drawable in ~3 minutes.

STAFF Q1. Design a real-time fraud / anomaly-detection system for Xero payments & invoices.

Scope & metrics. Score every payment/invoice event in <100 ms p99 at peak (tens of thousands QPS during EOFY/payroll runs). Business metric: fraud-\$ caught vs false-positive friction; ML metric: precision@k and recall at a fixed alert budget; guardrail: block-rate and customer-complaint rate. I'd clarify whether we *block* inline or *flag* for review — that changes the latency budget and the cost of false positives.

Data & contracts. Event stream (payments, logins, device, invoice metadata), historical labels from disputes/chargebacks (weak, delayed, imbalanced <0.5%). The research→prod seam is a **feature contract**: the exact features and transforms used in training must be recomputed identically online via a shared feature store, or training-serving skew destroys precision. I'd use entity-keyed features (per-org, per-bank-account, per-device velocity/aggregations) materialized to a low-latency online store.

Architecture. Streaming feature pipeline + online store + low-latency model server, with rules-engine fallback and a human-review queue feeding labels back. Async heavy models run out-of-band; a fast model gates inline.



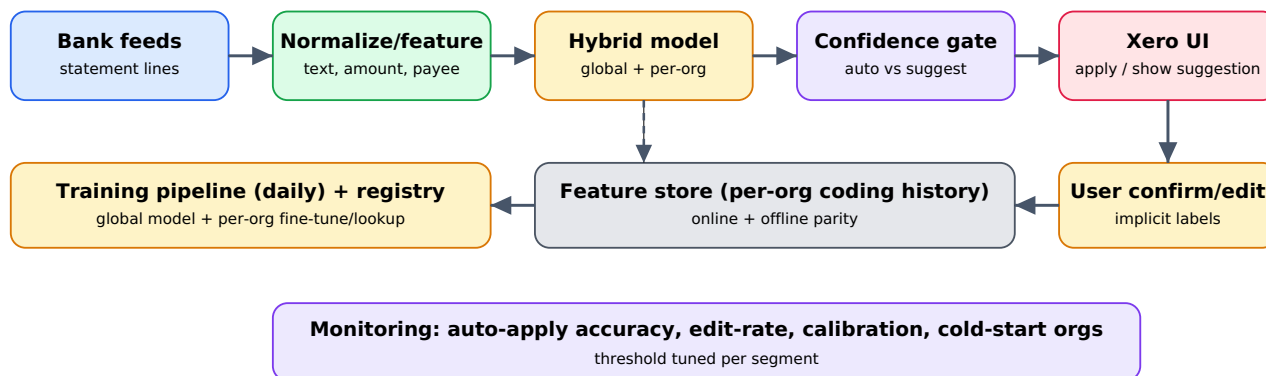
Fast inline GBM gate + async ensemble; analyst queue closes the label loop; shared offline/online feature store kills training-serving skew.

Production & trade-offs. Rules engine as deterministic fallback if the model server is unhealthy; canary new models in shadow mode and compare on a holdout before promotion; trigger retraining on PSI drift or recall decay. Biggest risk: label delay (chargebacks land weeks later) — I'd mitigate with analyst feedback as fast weak labels and reserve final metrics for matured labels.

SENIOR Q2. Design bank-transaction auto-categorization (statement line → account code) at Xero scale.

Scope & metrics. Billions of bank-feed lines map to a chart-of-accounts code per organization. It's multi-tenant and *personalized* — the same merchant maps differently per business. Latency is relaxed (near-real-time/batch on feed arrival, <1s acceptable); accuracy and **auto-apply confidence** matter most. Business metric: % of lines auto-coded correctly without user edits; ML metric: top-1 accuracy + calibrated confidence so we only auto-apply above a threshold; guardrail: incorrect-auto-apply rate.

Data & contracts. Features from transaction text/memo, amount, payee, frequency, plus per-org historical coding behavior (the strongest signal). Labels come *free* from user confirmations/edits — a beautiful implicit-feedback loop. The seam: a per-org embedding/feature contract plus a global model; the prod side must reproduce the same text normalization used in training.



Personalization via a per-org feature store; user edits are free labels; calibrated confidence decides auto-apply vs suggestion.

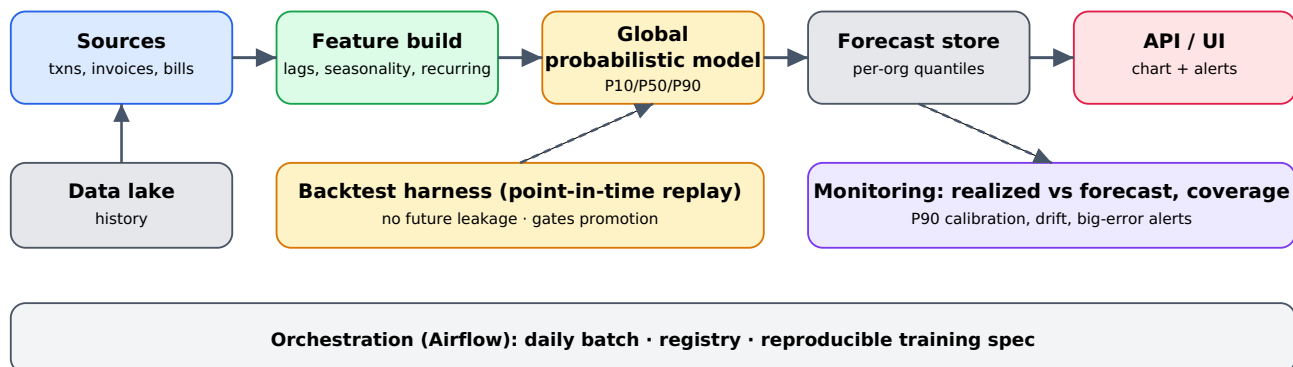
Production & trade-offs. Cold-start orgs lean on the global model and merchant priors; established orgs lean on personal history. I'd ship a v1 as global model + per-org nearest-neighbor lookup before investing in per-org fine-tuning. Calibration (isotonic/Platt) is essential because the threshold for

auto-apply is a business lever. Retraining is incremental/daily; monitor edit-rate per segment as the live quality signal.

SENIOR Q3. Design a short-term cash-flow forecasting system for small businesses.

Scope & metrics. Forecast each organization's bank balance / inflows-outflows 30-90 days ahead, refreshed daily. Batch (not real-time), but it runs for millions of orgs. Business metric: forecast usefulness / user trust; ML metric: pinball loss / MAPE at the chosen horizon and quantiles (P10/P50/P90), because users need *ranges*, not point estimates; guardrail: large-error rate on big accounts.

Data & contracts. Historical transactions, recurring invoices/bills (scheduled future events are gold), seasonality, and org metadata. The research→prod seam is a **backtesting contract**: the offline backtest harness must replay features exactly as they'd have been available at forecast time (no leakage of future data). I'd use a global probabilistic model (e.g., gradient-boosted quantile or a global DeepAR-style model) so small orgs borrow strength from the population.



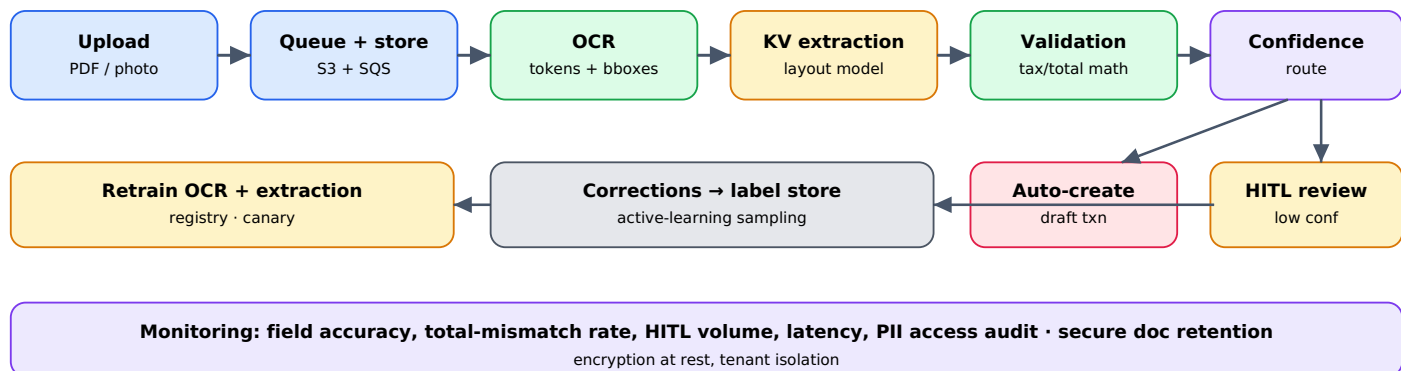
Probabilistic quantile forecasts; point-in-time backtest harness is the promotion gate; coverage/calibration are the live quality metrics.

Production & trade-offs. Ship a v1 with strong heuristics (recurring transactions + seasonality baseline) so you have a benchmark the ML must beat. Quantile coverage monitoring (does P90 actually contain 90% of outcomes?) is the trust metric. Trade-off: a global model generalizes but can miss idiosyncratic businesses — expose uncertainty bands honestly rather than over-promising point precision.

STAFF Q4. Design a document-understanding pipeline (receipts/invoices → structured fields), i.e. Hubdoc-style.

Scope & metrics. Users upload PDFs/photos; extract supplier, date, total, tax, line items, and push a draft transaction. Async (seconds to a minute acceptable), high volume, multilingual/multi-format. Business metric: % documents fully auto-extracted without correction; ML metric: field-level precision/recall and exact-match on totals (tax/total errors are unacceptable); guardrail: silent wrong-value rate.

Data & contracts. Pipeline = ingest → OCR → layout/key-value extraction model → business validation (does tax+net=total?) → confidence routing to human-in-the-loop. Labels come from user corrections. The seam: a clear contract between the OCR layer and the extraction model (token+bounding-box schema), so research can swap models without breaking downstream. PII and document retention/security are first-class here.



OCR→extraction→business validation; confidence routes to auto-create or human review; corrections drive active learning.

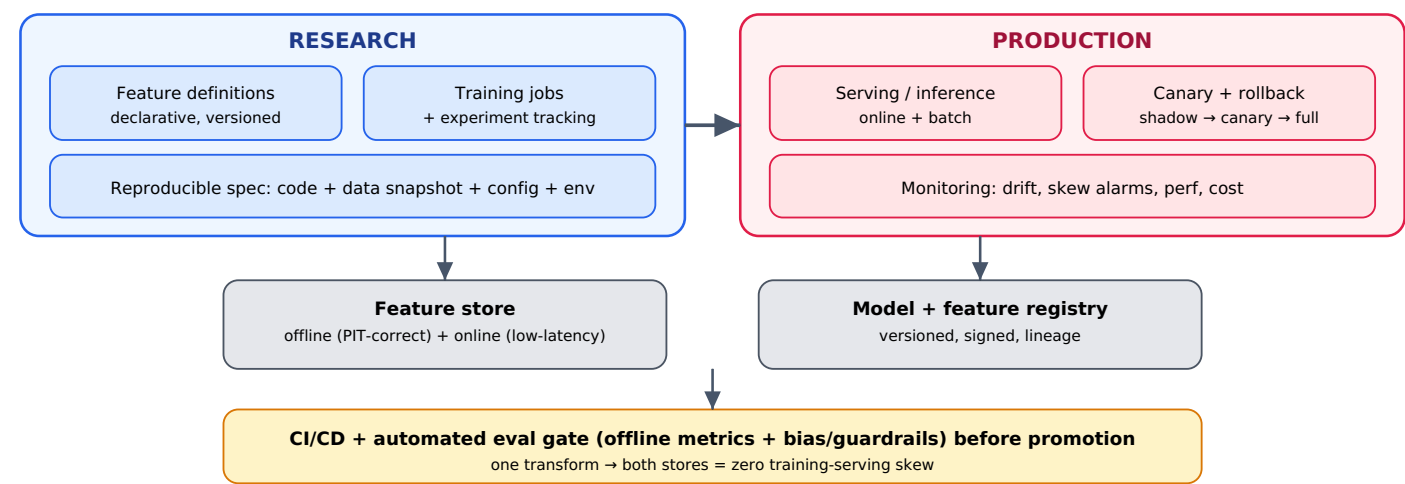
Production & trade-offs. Business-rule validation (arithmetic on totals/tax) catches model errors cheaply and is the single highest-ROI component — emphasize this. Route low-confidence or validation-failing docs to HITL, and sample those corrections via active learning. Decide build-vs-buy on OCR (managed OCR for v1, custom later). PII/security and tenant isolation are not optional at Xero.

STAFF Q5. Design Xero's ML platform: feature store + training-to-serving pipeline used by many teams.

Scope & metrics. This is the meta-question that directly probes the research→production boundary and your leadership. Goal: let many ML teams ship models safely and fast. Success metrics are platform metrics: time-to-first-model, training-serving skew incidents (target zero), rollback time, and reuse of features across teams. I'd clarify online vs offline needs, multi-tenancy, and governance/compliance requirements up front.

Data & contracts. The platform's entire value is **contracts**: a feature definition is written once and materialized to both an offline store (training, point-in-time correct) and an online store (serving, low latency) from the *same* transformation code — that's how you eliminate skew. A model registry holds versioned, signed artifacts with their training spec, data snapshot, and metrics. Promotion is gated by an automated eval + canary. CI/CD treats models

as deployable artifacts with rollback.



Platform = shared feature store (one transform, two stores) + signed model registry + automated eval gate + canary/rollback. This is the seam, productized.

Production & leadership. I'd sequence delivery: start with the registry + a thin online/offline store and one flagship model on it, prove skew elimination, then onboard teams. Govern with golden paths and paved roads rather than mandates. The leadership signal: you reduce org-wide risk and velocity by investing in the seam once instead of every team reinventing it.

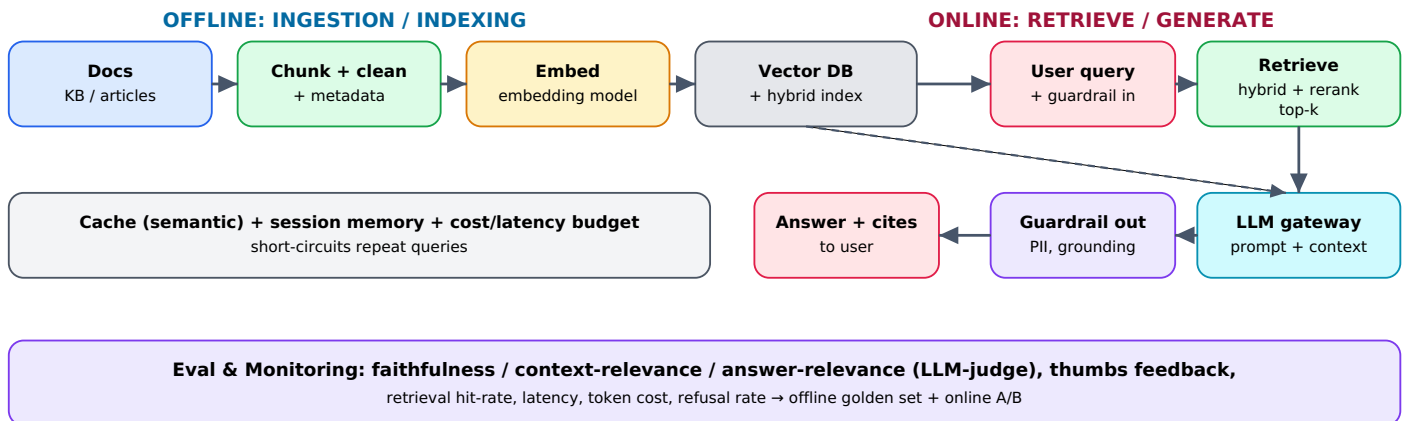
2 · Top 5 AI Agent / LLM / RAG System Design Questions

Same framing. Extra dimensions the interviewer probes here: retrieval quality, grounding/hallucination control, guardrails, eval (offline + online + LLM-as-judge), cost/latency of tokens, and the model-as-dependency (you don't own the model).

SENIOR Q1. Design a production RAG assistant over a large knowledge base (the generic Xbot pattern).

Scope & metrics. Answer user questions grounded in a document corpus, with citations, <3s p95 end-to-end. Business metric: deflection / self-serve resolution rate & CSAT; quality metrics: answer faithfulness (grounded, no hallucination), context relevance, answer relevance; guardrails: refusal-when-unknown rate, toxicity/PII leakage. I'd clarify corpus size, update frequency, and whether wrong answers are merely unhelpful or actively harmful (changes the grounding bar).

Data & contracts. Offline **ingestion/indexing** pipeline (chunk → embed → upsert to vector DB) is separate from the online **retrieve→generate** path. The seam between research and prod is the **prompt + retrieval contract**: chunking strategy, embedding model version, top-k, and prompt template are versioned artifacts; changing any requires re-eval. Treat the LLM as an external dependency behind a gateway so you can swap/upgrade models safely.



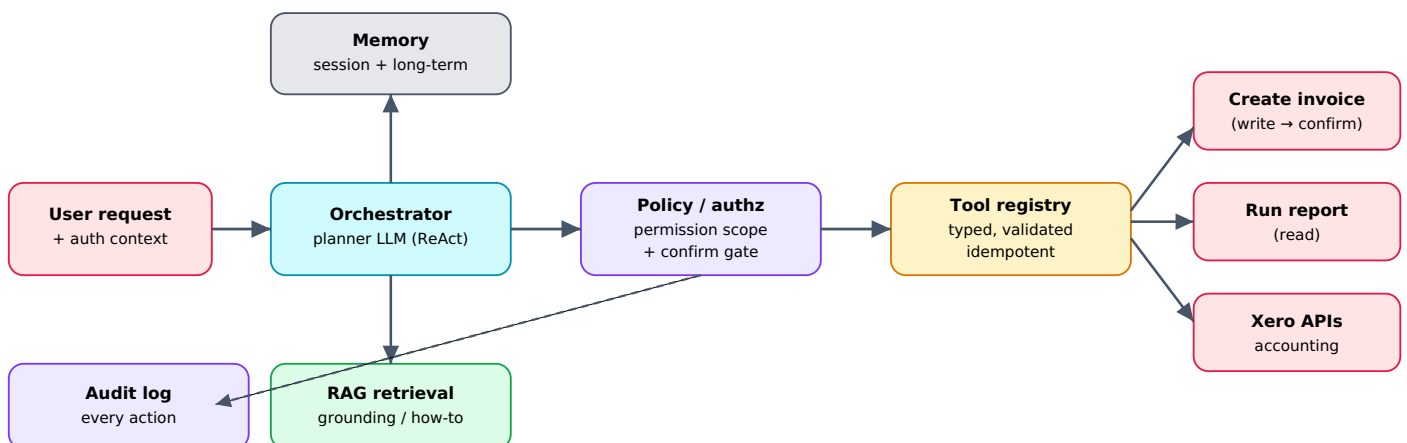
Two pipelines: offline indexing vs online retrieve→generate. Guardrails wrap both ends; LLM sits behind a gateway; eval runs offline (golden set) + online (feedback).

Production & trade-offs. Hybrid retrieval (dense + keyword) + a reranker beats dense-only on recall and precision. Semantic cache cuts cost/latency on repeat questions. Always cite sources and force "I don't know" when retrieval confidence is low — a confident wrong answer is the worst outcome. Version everything (chunking, embeddings, prompt) and re-run the golden-set eval on every change.

STAFF Q2. Design a multi-tool agentic assistant that can take actions in Xero (create invoice, run report).

Scope & metrics. Beyond Q&A: the agent plans and calls tools/APIs to *do* things. Stakes jump because actions mutate financial data. Business metric: task completion rate; quality: tool-selection accuracy, plan correctness, and — critically — **zero unauthorized/erroneous mutations**. I'd insist on human confirmation for any write/financial action and tight permission scoping.

Data & contracts. Architecture is an **orchestrator** (planner LLM (ReAct)) + a registry of typed **tools** (each a validated API wrapper with a schema), short/long-term **memory**, and a **policy/guardrail layer** that authorizes actions against the user's permissions. The seam: tool schemas are the contract — strongly typed, validated, idempotent, and least-privilege. Every action is logged for audit.



Planner LLM orchestrates typed tools; a policy/authz + confirm gate sits before any write; everything is audited. Memory + RAG inform the plan.

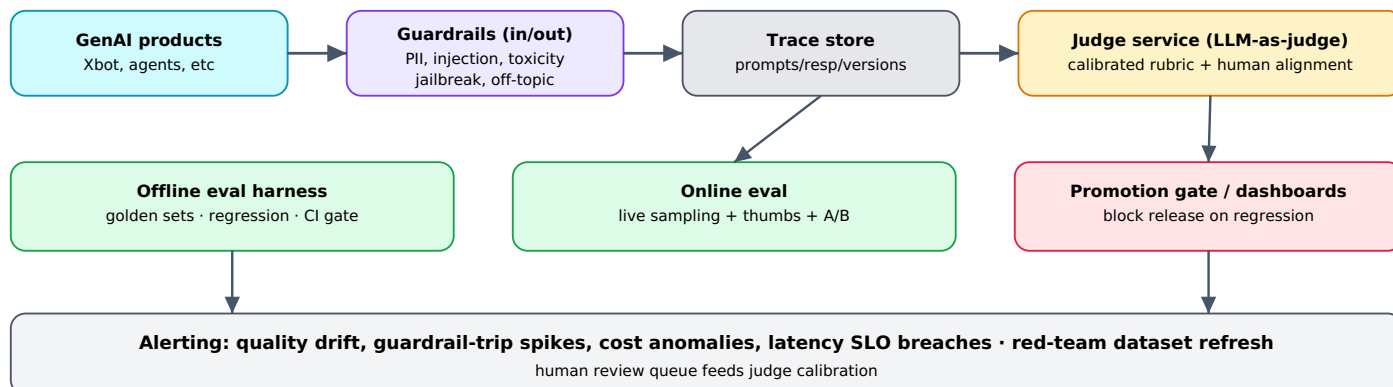
Production & trade-offs. Constrain the agent: small set of well-scoped tools beats an open-ended agent; cap reasoning loops to bound cost/latency; make tools idempotent so retries are safe. Human-in-the-loop confirmation on writes is the key safety lever for financial actions. Eval with task-level

success on a scenario suite plus tool-call trace inspection, not just final-answer judging.

STAFF Q3. Design an LLM evaluation, guardrails & observability platform shared across Xero's GenAI products.

Scope & metrics. A horizontal platform so every GenAI feature ships safely. It must support **offline eval** (golden datasets, regression on every prompt/model change), **online eval** (live sampling + LLM-as-judge + human review), and **inline guardrails** (PII, prompt-injection, toxicity, jailbreak, off-topic). Success metric: caught-regression rate before release; guardrail block precision/recall; mean time to detect a quality regression in prod.

Data & contracts. Central **trace store** capturing every prompt/response/retrieval/tool call with versions attached. Eval harness runs the golden set on each candidate config and gates promotion (the research→prod seam, again). A judge service (LLM-as-judge with calibrated rubrics + periodic human alignment) scores faithfulness/relevance/safety. Guardrails run as pre- and post-filters in the request path.



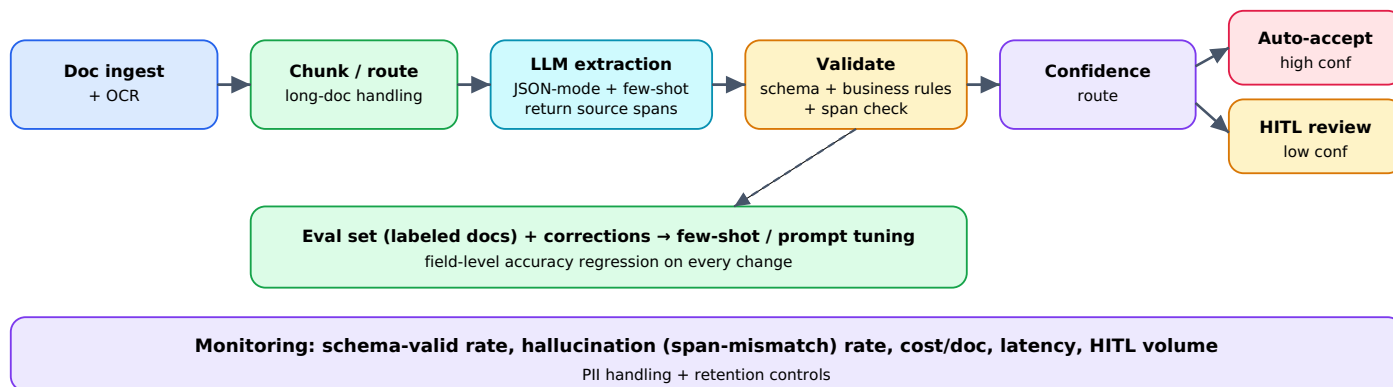
Horizontal platform: guardrails in the path, traces captured, offline + online eval feeding a promotion gate. Judge calibrated against human review.

Production & leadership. The leadership message: GenAI without an eval harness is shipping blind. Make offline eval a required CI gate and online judge-sampling the early-warning system. Maintain a versioned red-team/adversarial set and refresh it. Trade-off: LLM-judge is cheap but biased — anchor it to periodic human labels so the judge itself doesn't drift.

SENIOR Q4. Design an LLM-based document-to-structured-data extraction service (e.g., complex contracts/invoices).

Scope & metrics. Use an LLM (often multimodal) to extract structured JSON from messy documents where rule/layout models struggle. Async, high accuracy on key fields. Metric: field-level exact-match + schema-valid output rate; guardrail: hallucinated-field rate (LLM inventing values not in the doc). Clarify volume and cost ceiling — LLM tokens on long documents get expensive fast.

Data & contracts. Pipeline: ingest → (OCR if needed) → chunk/route → LLM extraction with a **strict output schema** (constrained/JSON-mode) → validation against the schema and business rules → confidence routing to HITL. The contract is the JSON schema + few-shot examples, versioned. Grounding: ask the model to return source spans so every extracted value is traceable, which makes hallucination detectable.



Constrained JSON output + source-span return makes hallucination detectable; validation + confidence routing protects accuracy; corrections refine prompts.

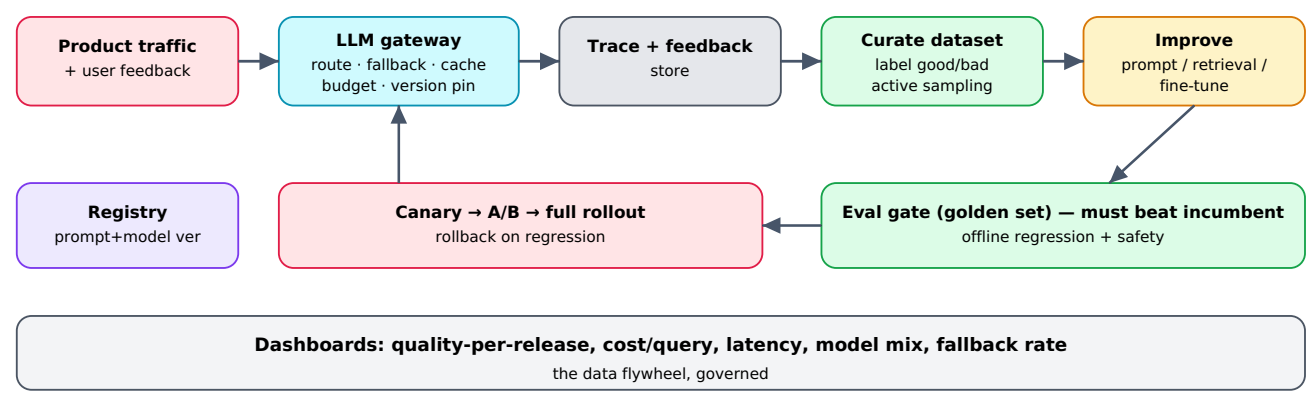
Production & trade-offs. Constrained decoding / JSON-mode guarantees parseable output; span-grounding catches invented values. Cost control: route easy docs to a cheaper model and only escalate hard ones (model cascade). Compare against a non-LLM extractor where rules suffice — don't pay LLM tokens for structured forms a cheap model handles. Keep a labeled regression set; field-level accuracy is the gate.

STAFF Q5. Design a continuous-improvement & fine-tuning pipeline (LLM gateway, feedback → eval → better prompts/models).

Scope & metrics. How a GenAI product improves over time without regressing. Centers on an **LLM gateway** (routing, fallback, caching, cost/rate control, version pinning) and a **data flywheel**: collect production traces + feedback → curate → eval → improve prompts, retrieval, or fine-tune → gated re-deploy. Metric: quality trend over releases with zero regression incidents; cost-per-resolved-query trend.

Data & contracts. The gateway is the prod-side contract: all GenAI traffic flows through it, so you can swap models, A/B prompts, enforce budgets, and capture traces uniformly. The flywheel's seam is the curated dataset + eval gate: nothing reaches prod without beating the incumbent on the golden

set. Fine-tuning (or preference tuning) is one lever among prompt/retrieval improvements — often not the first.



Gateway centralizes routing/cost/versioning; production feedback feeds a curated dataset; an eval gate ensures every change beats the incumbent before canary.

Production & leadership. Lead with prompt/retrieval improvements (cheap, fast, reversible) before fine-tuning (expensive, slower, but lowers cost/latency once stable). The gateway is the strategic investment — it decouples you from any single model vendor and gives uniform observability. The eval gate is what makes "continuous improvement" safe rather than continuous regression.

3 · The Main Question: Design "Xbot" for Xero

Problem statement (as the interviewer will give it):
"Xbot is a Generative AI API that answers Xero customer queries using content from Xero support articles. Design a RAG + LLM system that uses **Qdrant** as the vector store. Articles from **Xero Central** are indexed into a Qdrant collection. Include monitoring and evaluation."

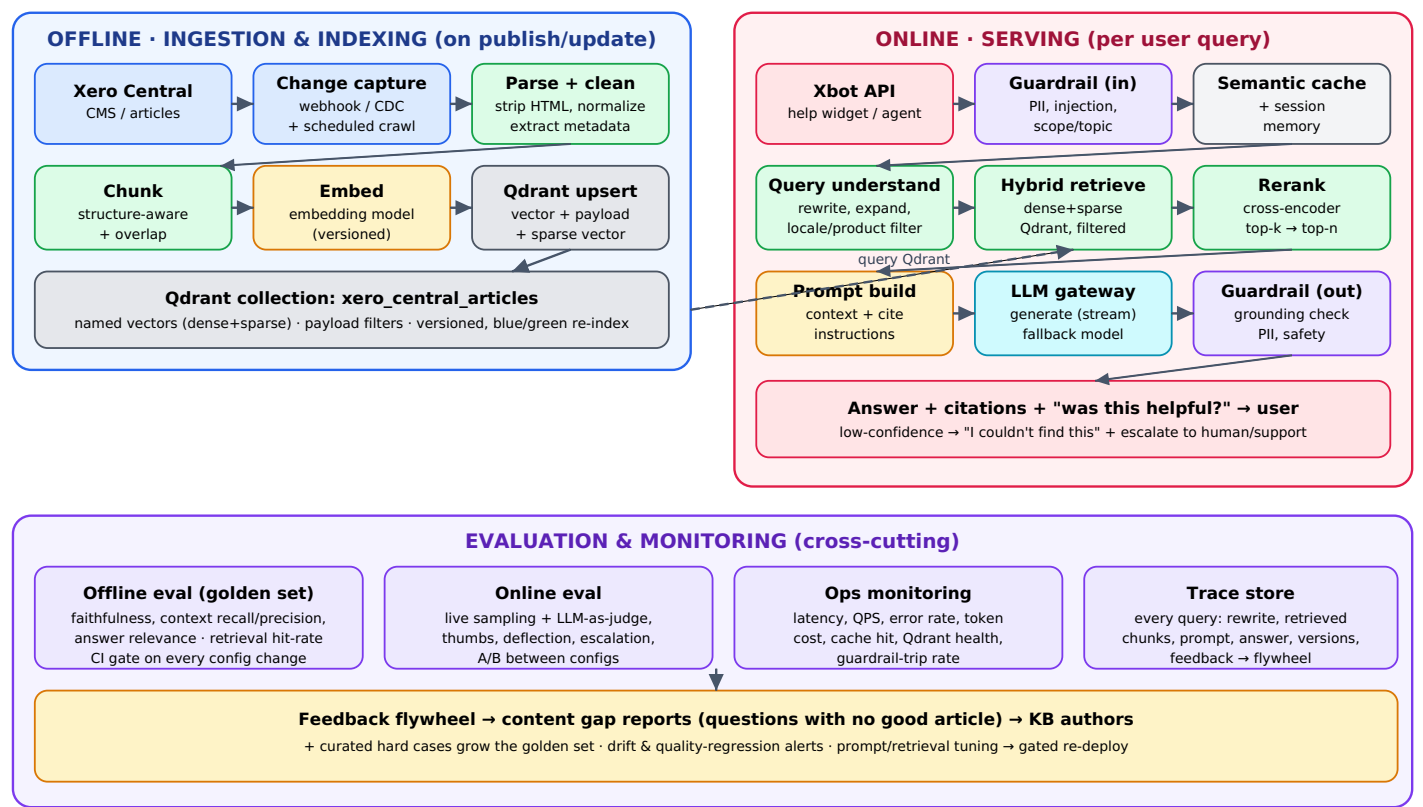
3.1 Scope, success metrics & SLOs (say this first)

Before drawing, I frame it. **Goal:** deflect support contacts by answering customer questions grounded *only* in Xero Central content, with citations, so answers are trustworthy and auditable. **Consumers:** in-product help widget, support agents (assist mode), and possibly the public help site. I'd ask: corpus size and update cadence (Xero Central is large and changes constantly), supported locales/regions, and whether wrong answers are a compliance risk (tax/accounting guidance can be). Then I commit to targets:

Dimension	Target / decision
Latency	< 3s p95 end-to-end (streamed first token < 1s for UX)
Grounding	Answer strictly from retrieved Xero Central context; cite article(s); refuse / escalate when unsupported
Quality (offline)	Faithfulness, context relevance, answer relevance (RAGAS-style) on a curated golden set
Quality (online)	Self-serve resolution / deflection rate, thumbs-up rate, escalation-to-human rate, CSAT
Guardrails	PII redaction, prompt-injection resistance, no advice outside KB scope, jurisdiction-correct content
Freshness	New/updated articles reflected in the index within minutes-hours of publish

3.2 End-to-end architecture

Two clearly separated planes — an **offline ingestion/indexing** plane (left, runs on article publish/update) and an **online serving** plane (right, runs per user query) — plus a cross-cutting **evaluation & monitoring** plane underneath. This separation is the headline of the whiteboard.



Xbot end-to-end: offline indexing into Qdrant (left), online retrieve→rerank→generate with guardrails (right), and a cross-cutting eval/monitoring/flywheel plane (bottom).

3.3 Ingestion & indexing pipeline (detail)

Triggered by Xero Central publish/update events (webhook/CDC) with a nightly reconciliation crawl as a safety net. Steps: **(1)** fetch & parse article, strip navigation/HTML, normalize whitespace; **(2)** extract metadata — article ID, title, URL, product (e.g., Payroll, Invoicing), region/locale, last-updated, category; **(3) structure-aware chunking** that respects headings/steps so a chunk is a coherent unit (target ~300-600 tokens with ~15% overlap), carrying parent-article metadata on every chunk; **(4)** embed each chunk with a versioned embedding model; **(5)** upsert into Qdrant with both a dense vector and a sparse (keyword) vector plus the metadata payload. Deletions/updates are handled by deterministic point IDs (hash of article+chunk) so re-indexing is idempotent. Embedding-model upgrades use a **blue/green re-index** into a new collection, validated by eval, then atomically aliased.

3.4 Qdrant collection design

This is where Qdrant-specific knowledge earns points. One collection **xero_central_articles**, point = one chunk, using **named vectors** for hybrid search and a rich payload for metadata filtering.

Element	Design choice & rationale
Point ID	Deterministic UUID from hash(article_id + chunk_index) → idempotent upserts, easy update/delete on article change.
Named vectors	dense (e.g., 1024-d, cosine) for semantic + sparse (SPLADE/BM25-style) for exact keyword/product-name matching. Qdrant supports both in one collection for hybrid search.
Distance	Cosine for the dense vector (normalize embeddings).
Payload	article_id, title, url, product, locale/region, category, last_updated, chunk_text, heading_path.
Payload indexes	Indexed keyword fields on product, locale, category so metadata filters are fast (e.g., restrict to the user's region/product). Critical for correctness — never serve AU payroll content to a UK user.
HNSW config	Tune m / ef_construct for recall vs build time; raise ef at search time for higher recall on hard queries.
Quantization	Scalar/binary quantization to cut memory/cost at scale, with rescoring on full vectors to preserve precision.
Operations	Snapshots for backup; collection aliases for zero-downtime re-index swaps; replication/sharding for HA + throughput.

Hybrid search in Qdrant: run dense + sparse queries and fuse (Qdrant's Query API / RRF fusion), *with a payload filter* for product+locale, then pass the fused top-k to a cross-encoder reranker before sending the top-n to the LLM. Dense alone misses exact product/feature names; sparse alone misses paraphrases. Fusion + rerank is the quality unlock.

Chunking & embedding strategy

- **Structure-aware** splits on headings/numbered steps so a "how-to" stays intact.
- ~300-600 tokens, ~15% overlap to avoid cutting answers mid-thought.
- Prepend article title/heading path to each chunk to preserve context.
- Embedding model is **versioned**; version stored in payload so retrieval never mixes embedding spaces.
- Evaluate chunk size/overlap empirically against the golden set — it's a tunable, not a guess.

Retrieval & prompt strategy

- Query rewrite/expansion (resolve "it"/follow-ups using session memory).
- Mandatory **metadata filter**: product + locale + (optionally) the user's plan.
- Hybrid dense+sparse → RRF fuse → cross-encoder rerank → top-n (3-6) chunks.
- Prompt instructs: answer **only** from provided context, cite article URLs, say "I don't know / here's how to contact support" if context is insufficient.
- Stream the response; attach citations the user can click through to Xero Central.

3.5 Guardrails & safety

Input: PII detection/redaction, prompt-injection detection, and topic-scope check (politely refuse non-Xero or out-of-scope questions). **Output:** a **grounding/faithfulness check** (does the answer's content trace to retrieved chunks? flag/suppress if not), PII scan, and a safety filter. For tax/accounting-adjacent answers, constrain to KB content and avoid generating bespoke financial advice. When retrieval confidence or grounding is low, Xbot must **degrade gracefully**: return "I couldn't find a confident answer in Xero Central" with the closest articles and a path to a human — a wrong confident answer is far costlier than an honest miss.

3.6 Evaluation (offline + online)

Layer	What we measure	How
Retrieval (offline)	Context precision/recall, hit-rate@k, MRR	Golden set of (question → known-relevant article/chunk) pairs curated from real support tickets + SME review.
Generation (offline)	Faithfulness/groundedness, answer relevance, citation correctness	RAGAS-style metrics with LLM-as-judge, calibrated against human ratings; run as a CI gate on every change to chunking, embeddings, prompt, model, or top-k.
Online (live)	Deflection/self-serve resolution, thumbs-up rate, escalation rate, CSAT, no-answer rate	Sampled live traffic scored by LLM-judge + periodic human review; A/B test config changes; guardrail-trip and refusal monitoring.
Business	Support-contact reduction, cost per resolved query	Compare cohorts with/without Xbot; track token + infra cost per answer.

Golden-set discipline: seed it from real Xero Central support questions, expand it with every production failure (hard cases caught online flow back into offline regression). The eval gate is non-negotiable: no config reaches prod without beating the incumbent on faithfulness and retrieval hit-rate.

3.7 Monitoring & the feedback flywheel

Operational: p50/p95/p99 latency per stage (retrieve, rerank, generate), QPS, error/timeout rates, token cost, cache-hit rate, Qdrant health (memory, search latency, indexing lag), and guardrail-trip rates — with SLO-based alerting. **Quality:** rolling faithfulness/thumbs/deflection with drift alarms (e.g., thumbs-down spike after a model or content change). **Flywheel:** the trace store logs every query (rewrite, retrieved chunks, prompt, answer, versions, feedback). Questions with *no good retrieval* become a **content-gap report** back to Xero Central authors — Xbot improves the knowledge base, not just itself. Curated hard cases grow the golden set; prompt/retrieval improvements ship through the same gated, canary-then-rollout path with instant rollback.

3.8 Production concerns: scaling, cost, failure modes

Concern	Approach
Scaling	Stateless API autoscaled behind a load balancer; Qdrant sharded + replicated; reranker and LLM behind the gateway with autoscaling/queueing; semantic cache absorbs repeat questions.
Cost	Cache + smaller-model routing for easy queries; cap context size; quantize Qdrant vectors; stream to improve perceived latency without raising cost.

LLM dependency	Gateway abstracts the model → fallback model on outage, version pinning, A/B, budget/rate limits. Never call a vendor SDK directly from business logic.
Stale content	Event-driven re-index + nightly reconciliation; <code>last_updated</code> in payload lets us prefer fresh articles and detect staleness.
Hallucination	Strict grounding prompt + output grounding check + citations + low-confidence refusal.
Wrong-locale answers	Mandatory payload filter on region/product; this is a correctness guardrail, not an optimization.
Security/PII	Redact PII pre-LLM, tenant isolation, audit logging, encryption at rest/in transit, retention controls.

3.9 Delivery sequencing & likely follow-ups (leadership signal)

v1: single-locale, dense-only retrieval, managed embedding + LLM, basic prompt + citations, offline golden-set eval, thumbs feedback. **v2:** hybrid + reranker, semantic cache, online LLM-judge sampling, content-gap reporting. **v3:** multi-locale filtering, blue/green re-index automation, model routing/cost optimization, agentic actions (deep-link into the product). Be ready for these probes:

- **"How do you stop hallucination?"** → grounding prompt + output grounding check + citations + refuse-on-low-confidence; measure faithfulness.
- **"An article changed — how fast does Xbot reflect it?"** → event-driven idempotent re-index by point ID; minutes, with nightly reconciliation.
- **"How do you know it's good before launch?"** → offline RAGAS golden-set gate; then shadow/canary with online judge + thumbs.
- **"Why Qdrant features specifically?"** → named vectors for hybrid, payload indexes for locale/product filtering, quantization for cost, aliases/snapshots for safe re-index.
- **"Research↔prod boundary?"** → chunking + embedding-version + prompt template are versioned artifacts; the eval gate is the seam; the gateway decouples the model.

One-line summary to close the whiteboard: "Xbot is two pipelines and a feedback loop — an idempotent, event-driven indexer into a hybrid Qdrant collection; an online retrieve→rerank→grounded-generate path guarded at both ends and served behind an LLM gateway; and a cross-cutting eval + monitoring plane whose golden-set gate governs the research↔production seam and whose flywheel improves both Xbot and Xero Central itself."